Analysis and Design of Algorithms (ADA)

Lab Assignment – 3

Submitted by : Md. Anas khan

Submitted to : Dr. Aarti

Course : B.Tech – CSE (Data Science)

Roll no. : 2401420035

https://github.com/KLAWS5/graph-algo-mini-project--Md.-Anas-khan-

Graph Algorithms Assignment

```
import time,random,math
from collections import deque,defaultdict
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
random.seed(7);np.random.seed(7)
plt.rcParams['figure.figsize']=(7,4)
plt.rcParams['font.size']=10
```

1) Graph representation

```
nodes=['A','B','C','D','E','F']
edges=[('A','B'),('A','C'),('B','D'),('B','E'),('C','E'),('D','F'),('E','F')]
g={n:[] for n in nodes}
for u,v in edges:g[u].append(v);g[v].append(u)
mat=pd.DataFrame(0,index=nodes,columns=nodes)
for u,v in edges:mat.loc[u,v]=1;mat.loc[v,u]=1
print('Adjacency List:')
for k,v in g.items():print(k,':',v)
print('\nAdjacency Matrix:')
display(mat)
```

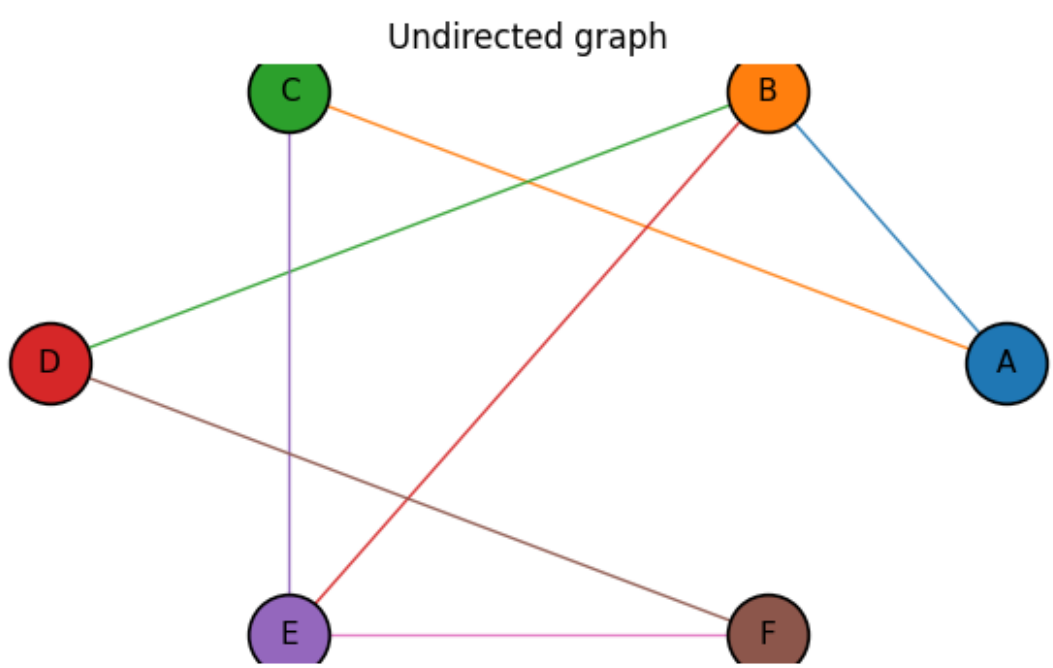
```
Adjacency List:
A : ['B', 'C']
B : ['A', 'D', 'E']
C : ['A', 'E']
D : ['B', 'F']
E : ['B', 'C', 'F']
F : ['D', 'E']

Adjacency Matrix:
```

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	0	1	1	0
C	1	0	0	0	1	0
D	0	1	0	0	0	1
E	0	1	1	0	0	1
F	0	0	0	1	1	0

Base graph :

```
def circle_pos(nodes,r=1.8,center=(0,0)):
    n=len(nodes);cx,cy=center
    return {nodes[i]:(cx+r*math.cos(2*math.pi*i/n),cy+r*math.sin(2*math.pi*i/n)) for i in
range(n)}
```

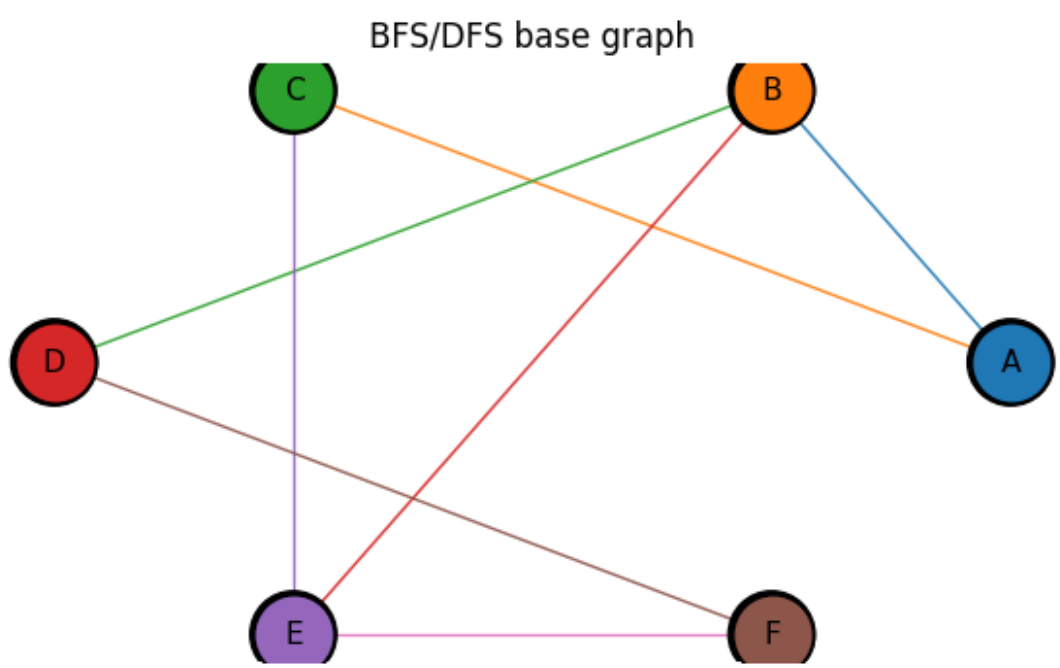
```
def
draw_graph(nodes,edges,title='',directed=False,pos=None,highlight_nodes=None,highlight_edges=
None,edge_labels=None):
    pos=pos or circle_pos(nodes)
    hn=set(highlight_nodes or [])
    he={tuple(e) for e in (highlight_edges or [])}
    fig,ax=plt.subplots()
    for u,v in edges:
        x1,y1=pos[u];x2,y2=pos[v]
        is_h=(u,v) in he or (not directed and (v,u) in he)
        if directed:
            ax.annotate('',xy=(x2,y2),xytext=(x1,y1),arrowprops=dict(arrowstyle='->',lw=2 if
is_h else 1,alpha=.9))
        else:
            ax.plot([x1,x2],[y1,y2],lw=2 if is_h else 1,alpha=.9)
        if edge_labels and (u,v) in edge_labels:
            ax.text((x1+x2)/2,(y1+y2)/2,str(edge_labels[(u,v)]),fontsize=9)
        elif edge_labels and (not directed) and (v,u) in edge_labels:
            ax.text((x1+x2)/2,(y1+y2)/2,str(edge_labels[(v,u)]),fontsize=9)
    for n in nodes:
        x,y=pos[n]
        ax.scatter([x],[y],s=900,zorder=3,edgecolors='black',linewidths=1.2)
        ax.text(x,y,n,ha='center',va='center',fontsize=11,zorder=4)
        if n in hn:
ax.scatter([x],[y],s=900,zorder=2,facecolors='none',edgecolors='black',linewidths=3)
    ax.set_title(title)
    ax.axis('off')
    plt.show()
draw_graph(nodes,edges,'Undirected graph') draw_graph(nodes,edges,'Undirected graph')
```



2) BFS, DFS, and friend suggestions

```
def bfs(graph,start):
    vis={start};q=deque([start]);order=[]
    while q:
        u=q.popleft();order.append(u)
        for v in graph[u]:
            if v not in vis:vis.add(v);q.append(v)
    return order
def dfs(graph,start,vis=None,order=None):
    if vis is None:vis=set();order=[]
    vis.add(start);order.append(start)
    for v in graph[start]:
        if v not in vis:dfs(graph,v,vis,order)
    return order
def friend_suggestions(graph,user):
    direct=set(graph[user])|{user};c=defaultdict(int)
    for f in graph[user]:
        for fof in graph[f]:
            if fof not in direct:c[fof]+=1
    return dict(sorted(c.items(),key=lambda x:(-x[1],x[0])))
b=bfs(g,'A');d=dfs(g,'A');s=friend_suggestions(g,'A')
print('BFS order:',b)
print('DFS order:',d)
print('Friend suggestions for A:',s)
draw_graph(nodes,edges,'BFS/DFS base graph',highlight_nodes=b)
```

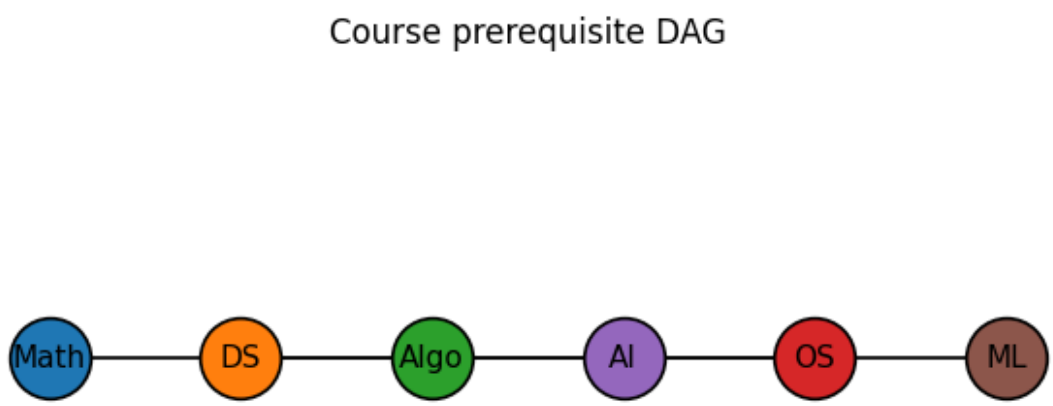
```
BFS order: ['A', 'B', 'C', 'D', 'E', 'F']
DFS order: ['A', 'B', 'D', 'F', 'E', 'C']
Friend suggestions for A: {'E': 2, 'D': 1}
```



3) Topological sorting

```
dag_nodes=['Math','DS','Algo','OS','AI','ML']
dag_edges=[('Math','DS'),('Math','Algo'),('DS','OS'),('Algo','OS'),('DS','AI'),('OS','ML'),('
AI','ML')]
dag={n:[] for n in dag_nodes}
indeg={n:0 for n in dag_nodes}
for u,v in dag_edges:dag[u].append(v);indeg[v]+=1

def topo_sort(graph,indeg):
    indeg=indeg.copy();q=deque([u for u in indeg if indeg[u]==0]);order=[]
    while q:
        u=q.popleft();order.append(u)
        for v in graph[u]:
            indeg[v]-=1
            if indeg[v]==0:q.append(v)
    return order

topo=topo_sort(dag,indeg)
print('Topological order:',topo)
pos={n:(i,0) for i,n in enumerate(topo)}
draw_graph(dag_nodes,dag_edges,'Course prerequisite DAG',directed=True,pos=pos)
```

```
Topological order: ['Math', 'DS', 'Algo', 'AI', 'OS', 'ML']
```



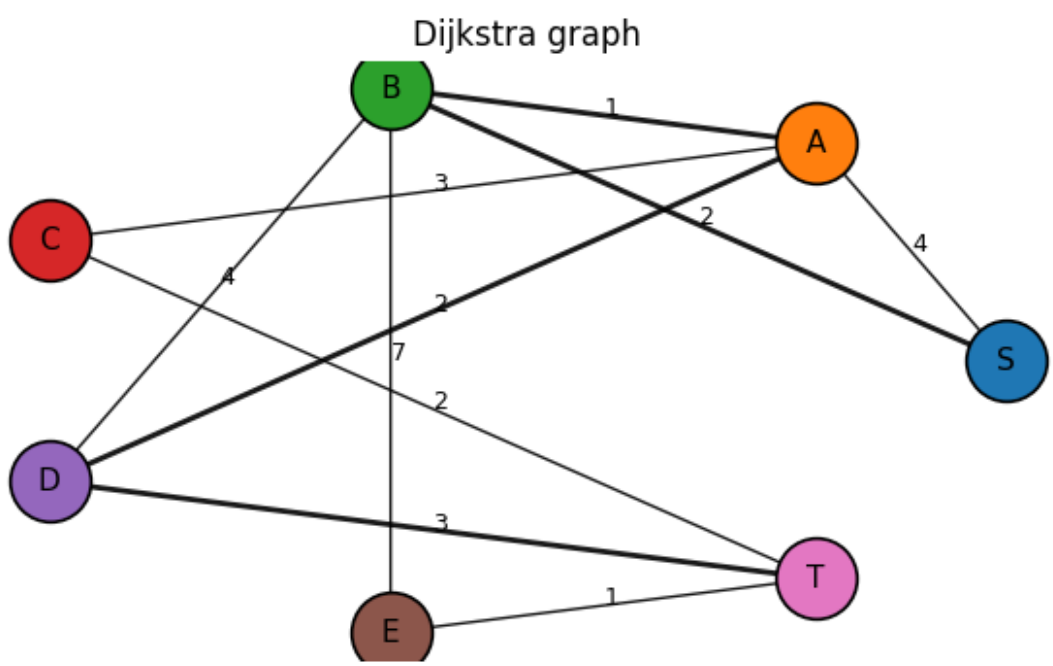
4) Shortest path algorithms

Dijkstra :

```
wg_nodes=['S','A','B','C','D','E','T']
wg_edges=[('S','A',4),('S','B',2),('A','C',3),('A','D',2),('B','A',1),('B','D',4),('B','E',7)
,('C','T',2),('D','T',3),('E','T',1)]
```

```python
wg={n:{} for n in wg_nodes}
for u,v,w in wg_edges:
    wg[u][v]=w

def dijkstra(graph,src):
    dist={n:math.inf for n in graph};prev={};dist[src]=0;seen=set()
    while len(seen)<len(graph):
        u=min((n for n in graph if n not in seen),key=lambda n:dist[n],default=None)
        if u is None or dist[u]==math.inf:break
        seen.add(u)
        for v,w in graph[u].items():
            if dist[u]+w<dist[v]:dist[v]=dist[u]+w;prev[v]=u
    return dist,prev

def path(prev,t):
    p=[t]
    while t in prev:t=prev[t];p.append(t)
    return p[::-1]

dist,prev=dijkstra(wg,'S')
p=path(prev,'T')
print('Distances from S:',dist)
print('Shortest path S -> T:',p,' cost =',dist['T'])
draw_graph(wg_nodes,[(u,v) for u,v,w in wg_edges],'Dijkstra
graph',directed=True,highlight_edges=list(zip(p,p[1:])),edge_labels={(u,v):w for u,v,w in
wg_edges})
```

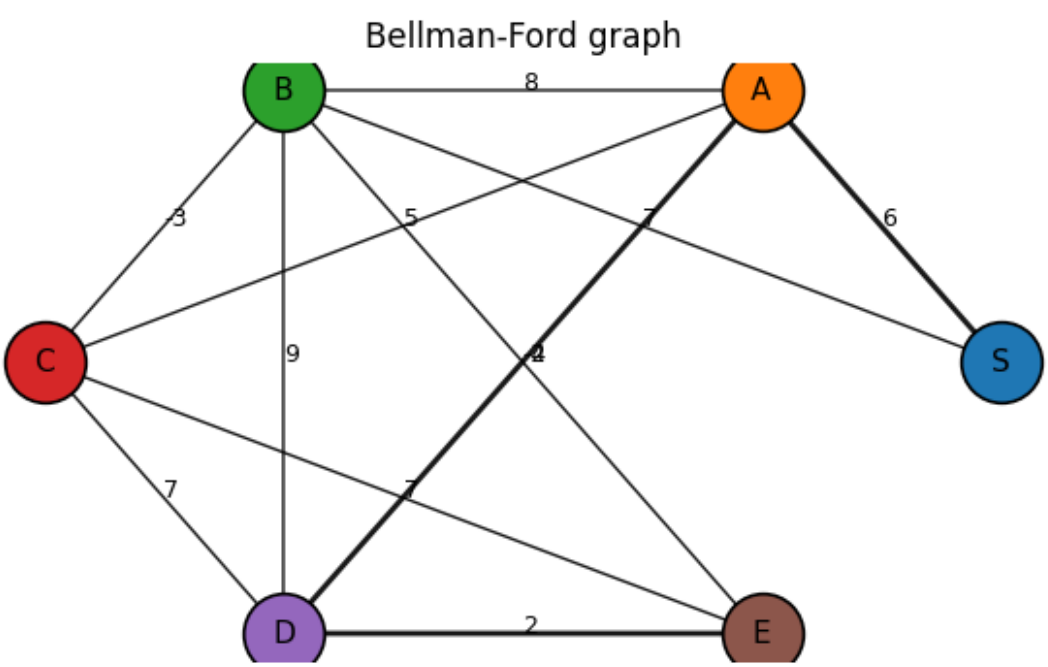
```
Distances from S: {'S': 0, 'A': 3, 'B': 2, 'C': 6, 'D': 5, 'E': 9, 'T': 8}
Shortest path S -> T: ['S', 'B', 'A', 'D', 'T']  cost = 8
```



Bellman ford :

```
bf_nodes=['S','A','B','C','D','E']
bf_edges=[('S','A',6),('S','B',7),('A','C',5),('A','D',-4),('A','B',8),('B','C',-
3),('B','D',9),('C','E',7),('D','C',7),('D','E',2),('E','B',-2)]
bf={n:{} for n in bf_nodes}
for u,v,w in bf_edges:bf[u][v]=w

def bellman_ford(graph,src):
    dist={n:math.inf for n in graph};prev={};dist[src]=0
    edges=[(u,v,w) for u in graph for v,w in graph[u].items()]
    for _ in range(len(graph)-1):
        upd=False
        for u,v,w in edges:
            if dist[u]!=math.inf and dist[u]+w<dist[v]:
                dist[v]=dist[u]+w;prev[v]=u;upd=True
        if not upd:break
    neg=any(dist[u]!=math.inf and dist[u]+w<dist[v] for u,v,w in edges)
    return dist,prev,neg
dist2,prev2,neg=bellman_ford(bf,'S')
print('Bellman-Ford distances from S:',dist2)
print('Negative cycle detected:',neg)
print('Path to E:',path(prev2,'E'),' cost =',dist2['E'])
draw_graph(bf_nodes,[(u,v) for u,v,w in bf_edges],'Bellman-Ford
graph',directed=True,highlight_edges=list(zip(path(prev2,'E'),path(prev2,'E')[1:])),edge_labe
ls={(u,v):w for u,v,w in bf_edges})
```

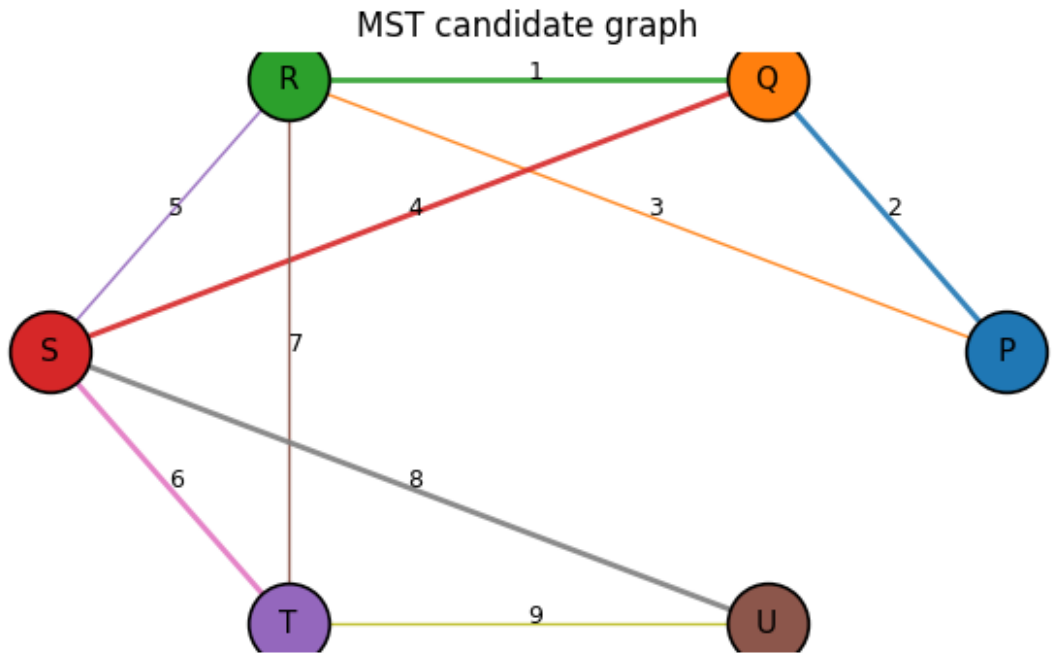
```
Bellman-Ford distances from S: {'S': 0, 'A': 6, 'B': 2, 'C': -1, 'D': 2, 'E': 4}
Negative cycle detected: False
Path to E: ['S', 'A', 'D', 'E']  cost = 4
```



5) Minimum spanning tree

```
mst_nodes=['P','Q','R','S','T','U']
mst_edges=[('P','Q',2),('P','R',3),('Q','R',1),('Q','S',4),('R','S',5),('R','T',7),('S','T',6
),('S','U',8),('T','U',9)]
mst_adj={n:{} for n in mst_nodes}
for u,v,w in mst_edges:mst_adj[u][v]=w;mst_adj[v][u]=w

def prim(graph,start):
    vis={start};res=[];total=0
    while len(vis)<len(graph):
        cand=min((w,u,v) for u in vis for v,w in graph[u].items() if v not in vis)
        w,u,v=cand;vis.add(v);res.append((u,v,w));total+=w
    return res,total

def kruskal(nodes,edges):
    parent={n:n for n in nodes}
    def find(x):
        while parent[x]!=x:
            parent[x]=parent[parent[x]]
            x=parent[x]
        return x
    def union(a,b):parent[find(a)]=find(b)
    res=[];total=0
    for w,u,v in sorted((w,u,v) for u,v,w in edges):
        if find(u)!=find(v):union(u,v);res.append((u,v,w));total+=w
    return res,total

pm,pt=prim(mst_adj,'P')
km,kt=kruskal(mst_nodes,mst_edges)
print('Prim MST:',pm,' total =',pt)
print('Kruskal MST:',km,' total =',kt)
draw_graph(mst_nodes,[(u,v) for u,v,w in mst_edges],'MST candidate
graph',highlight_edges=[(u,v) for u,v,w in pm],edge_labels={(u,v):w for u,v,w in mst_edges})
```

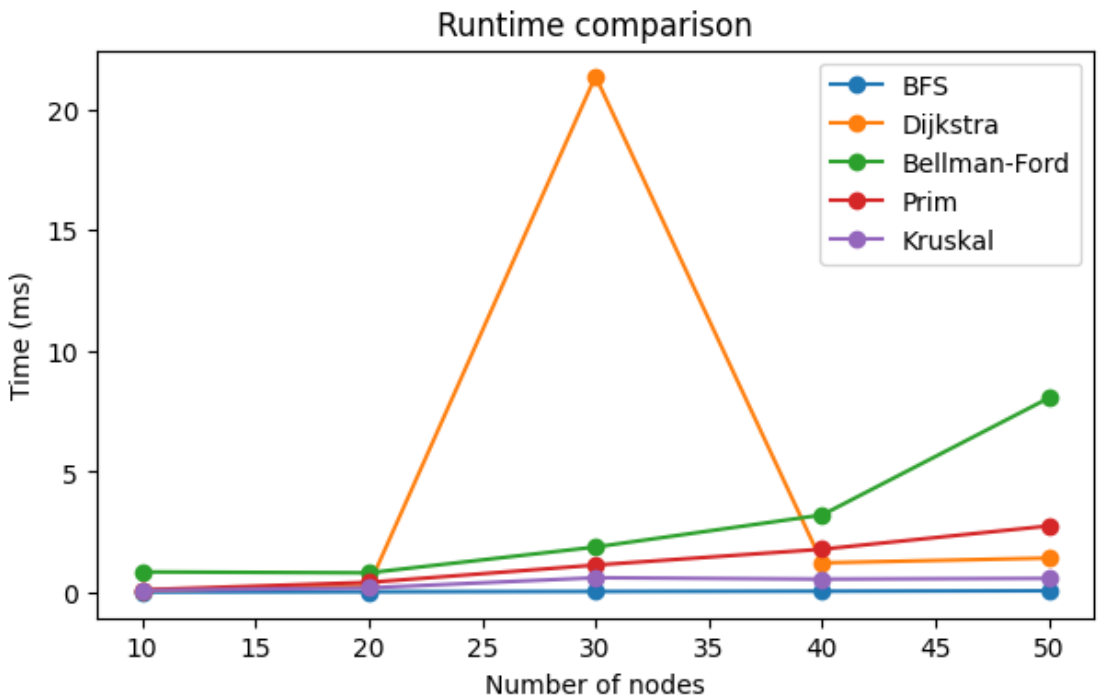
```
Prim MST: [('P', 'Q', 2), ('Q', 'R', 1), ('Q', 'S', 4), ('S', 'T', 6), ('S', 'U', 8)]
total = 21
Kruskal MST: [('Q', 'R', 1), ('P', 'Q', 2), ('Q', 'S', 4), ('S', 'T', 6), ('S', 'U', 8)]
total = 21
```



6) Time comparison on random graphs

```
def rnd_graph(n,m,weighted=False,directed=False,neg=False):
    nodes=list(range(n))
    edges=set()
    while len(edges)<m:
        u,v=random.sample(nodes,2)
        if not directed and u>v:u,v=v,u
        edges.add((u,v))
    if weighted:
        if directed:
            g={i:{} for i in nodes}
            for u,v in edges:
                w=random.randint(1,9)
                if neg and random.random()<0.25:w*=-1
                g[u][v]=w
            return g
        g={i:{} for i in nodes}
        for u,v in edges:
            w=random.randint(1,9)
            if neg and random.random()<0.25:w*=-1
            g[u][v]=w;g[v][u]=w
        return g
    g={i:[] for i in nodes}
    for u,v in edges:
        g[u].append(v)
        if not directed:g[v].append(u)
    return g

sizes=[10,20,30,40,50]
times={'BFS':[],'Dijkstra':[],'Bellman-Ford':[],'Prim':[],'Kruskal':[]}
for n in sizes:
```

```
    m=n*3
    G1=rnd_graph(n,m)
    G2=rnd_graph(n,m,weighted=True)
    G3=rnd_graph(n,m,weighted=True,directed=True,neg=True)
    G4=rnd_graph(n,m,weighted=True)
    s0=0
    t=time.perf_counter();bfs(G1,s0);times['BFS'].append((time.perf_counter()-t)*1000)
    t=time.perf_counter();dijkstra(G2,s0);times['Dijkstra'].append((time.perf_counter()-
t)*1000)
    t=time.perf_counter();bellman_ford(G3,s0);times['Bellman-
Ford'].append((time.perf_counter()-t)*1000)
    t=time.perf_counter();prim(G4,s0);times['Prim'].append((time.perf_counter()-t)*1000)
    t=time.perf_counter();kruskal(list(G4.keys()),[(u,v,w) for u in G4 for v,w in
G4[u].items() if u<v]);times['Kruskal'].append((time.perf_counter()-t)*1000)

for name,vals in times.items():
    plt.plot(sizes,vals,marker='o',label=name)
plt.xlabel('Number of nodes');plt.ylabel('Time (ms)');plt.title('Runtime comparison')
plt.legend();plt.show()
```



7) Summary

- **BFS/DFS** explore connectivity and friend recommendations.
- **Topological sort** handles task dependencies.
- **Dijkstra** finds shortest non-negative paths.
- **Bellman-Ford** supports negative edges and cycle detection.
- **Prim/Kruskal** build low-cost network layouts.

https://github.com/KLAWS5/graph-algo-mini-project--Md.-Anas-khan-